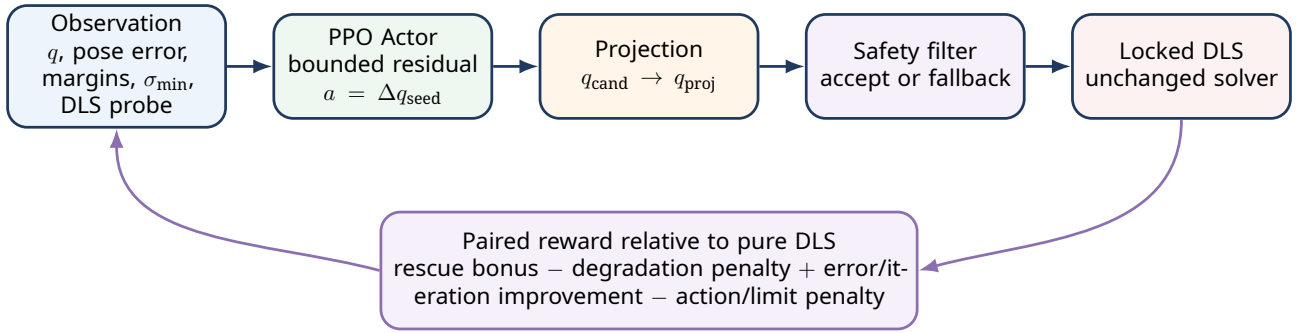


SR-PPO-DLS

Safe Residual PPO-Guided DLS Initializer

Phân tích lý thuyết, công thức toán học và phương án áp dụng
vào Dataset v2 của robot Kassow KR810



Robot: Kassow KR810, 7 bậc tự do

Phạm vi: numerical inverse kinematics ở mức động học; không có actuator, dynamics, collision hoặc robot thật

Baseline: Dataset v2.0.0 và locked DLS candidate `cand_D_pure_dls`

Ngày tổng hợp: 27/07/2026

Contents

1	Tóm tắt điều hành	1
2	Bài toán nghiên cứu và phạm vi	1
2.1	Bài toán IK của KR810	1
2.2	Phạm vi đúng của báo cáo	2
2.3	Giả thuyết nghiên cứu	2
3	Nền tảng toán học của DLS	2
3.1	Pose error	2
3.2	Jacobian và xấp xỉ cục bộ	3
3.3	Bài toán tối ưu DLS	3
3.4	Adaptive damping và giới hạn bước	3
3.5	Tại sao PPO cần cải thiện seed thay vì thay DLS	4
4	Nền tảng toán học của PPO	4
4.1	MDP cho one-shot initializer	4
4.2	Actor-Critic	4
4.3	PPO clipped objective	5
4.4	Advantage và value loss	5
5	Nguyên lý Residual và Safe trong SR-PPO-DLS	5
5.1	Residual reinforcement learning	5
5.2	Safety layer và action projection	6
5.3	Tại sao cần safety filter khi pure DLS đã mạnh	6
6	Kiến trúc SR-PPO-DLS được đề xuất	6
6.1	Sơ đồ tổng thể	6
6.2	Một episode one-shot	6
6.3	Observation 36 chiều	7
6.4	DLS probe direction	7
6.5	Action residual	7
6.6	Projection vào safe joint range	8
6.7	Safety filter hai tầng	8
7	Reward paired relative to pure DLS	9
7.1	Tại sao không dùng reward tuyệt đối	9
7.2	Success indicators	9
7.3	Reward đề xuất	9
7.4	Reward weights là hyperparameter, không phải kết luận lý thuyết	10
8	Quy trình huấn luyện PPO	10
8.1	Training stream và fixed benchmark	10
8.2	Sampling theo failure mode	10
8.3	PPO training loop	10
8.4	Cấu hình khởi đầu	11
8.5	TPE hyperparameter optimization	11
9	Áp dụng vào Dataset v2 hiện tại	12
9.1	Tài sản dữ liệu hiện có	12
9.2	Mapping field cho Point-IK environment	12
9.3	Vai trò của từng split	12
9.4	Primary application: Point-IK	13

9.5	Secondary application: trajectory waypoint 0	13
9.6	Tại sao không PPO ở mọi waypoint	13
10	Thiết kế thực nghiệm và đánh giá công bằng	13
10.1	Baselines bắt buộc	13
10.2	Point-IK metrics	14
10.3	Trajectory metrics	14
10.4	Paired statistics	15
10.5	Acceptance gates đề xuất	15
11	Truy vết: thành phần nào áp dụng từ bài báo nào?	15
12	Kiến trúc source code đề xuất	16
12.1	Test bắt buộc	17
13	Quy trình triển khai theo giai đoạn	17
14	Rủi ro, giới hạn và cách kiểm soát	17
14.1	One-step PPO có thể không cần critic mạnh	17
14.2	Reward hacking qua fallback	18
14.3	Memorization	18
14.4	Action scale quá nhỏ hoặc quá lớn	18
14.5	Safety filter quá bảo thủ	18
14.6	Không suy rộng sang robot thật	18
15	Kết luận thiết kế	18

1. Tóm tắt điều hành

Quyết định đã chốt

Phương pháp tiếp theo của project là **SR-PPO-DLS: Safe Residual PPO-Guided DLS Initializer**. PPO chỉ tạo một hiệu chỉnh khớp có biên giới hạn để chuyển q_{initial} sang một seed tốt hơn. Một safety filter kiểm tra proposal và có quyền quay về seed gốc. Sau đó, chính locked DLS hiện tại chịu trách nhiệm hội tụ tới pose cuối. Không phát triển Multistep PPO trong phạm vi phương pháp này.

Mục tiêu của SR-PPO-DLS không phải thay thế numerical IK. Mục tiêu là giải đúng một điểm yếu đã được Dataset v2 đo được: DLS rất nhạy với initial configuration và thường dừng vì stagnation trong local basin. Trên frozen-test, locked DLS đạt standard Point-IK success khoảng 95.08%, nhưng trajectory standard success chỉ khoảng 54.93% với warm-start và 32.92% với cold-start. Frozen run hoàn thành đủ 171,600 DLS solves và không có non-finite result, nên vấn đề không phải lỗi số học nền tảng mà là lựa chọn basin/seed và descent cục bộ [7, 8].

Phương pháp đề xuất kết hợp năm nguồn ý tưởng:

1. **PPO-Clip** cung cấp thuật toán học policy liên tục ổn định hơn policy-gradient cơ bản [2].
2. **MPDIK** cung cấp giả thuyết trung tâm: PPO có thể cải thiện initial value, còn DLS cung cấp độ chính xác IK cuối [1].
3. **Residual reinforcement learning** cung cấp cách phân rã: giữ bộ giải cổ điển mạnh và chỉ học phần hiệu chỉnh còn thiếu [3].
4. **Safety layer** cung cấp nguyên tắc chỉnh/projection action trước khi áp dụng vào hệ thống có ràng buộc [4].
5. **TPE cho PPO trên robot 7-DOF** cho thấy hyperparameter optimization có thể cải thiện đáng kể hiệu năng và tốc độ hội tụ; vì vậy action scale, learning rate, entropy và clip range không nên được chọn tùy ý [5].

Điểm cần phân biệt

Tên *Safe Residual PPO-Guided DLS Initializer* là một **kiến trúc tổng hợp mới**. Bài Yu và Tan dùng PPO xen kẽ DLS nhiều bước; Johannink et al. học residual của controller; Dalal et al. dùng safety layer dựa trên mô hình tuyến tính. Project KR810 chỉ kế thừa nguyên tắc và điều chỉnh cho one-shot seed selection, không tuyên bố đã tái tạo nguyên bản các thuật toán đó.

2. Bài toán nghiên cứu và phạm vi

2.1. Bài toán IK của KR810

Kassow KR810 có bảy biến khớp chủ động:

$$q = [q_1, q_2, \dots, q_7]^T \in \mathbb{R}^7. \quad (1)$$

Forward kinematics ánh xạ cấu hình khớp sang pose đầu công tác:

$$T(q) = \text{FK}(q) = \begin{bmatrix} R(q) & p(q) \\ 0 & 1 \end{bmatrix} \in \text{SE}(3), \quad (2)$$

trong đó $p(q) \in \mathbb{R}^3$ và $R(q) \in \text{SO}(3)$. Bài toán IK nhận pose đích $T_d = (R_d, p_d)$ và trạng thái ban đầu q_{initial} , sau đó tìm q^* sao cho:

$$\|p_d - p(q^*)\| \leq \varepsilon_p, \quad \theta(R_d R(q^*)^T) \leq \varepsilon_o. \quad (3)$$

Do task pose có sáu chiều trong khi robot có bảy khớp, KR810 là redundant đối với task 6D. Một pose có thể tương ứng nhiều nghiệm khớp. Vì vậy chất lượng của seed ảnh hưởng đến nhánh nghiệm, tốc độ hội tụ, khoảng cách tới joint limit và khả năng tránh local basin.

2.2. Phạm vi đúng của báo cáo

Báo cáo chỉ xét kinematics:

- target pose, FK, Jacobian và numerical IK;
- joint limits và singularity diagnostics;
- PPO học một residual seed;
- đánh giá Point-IK và ảnh hưởng tới trajectory warm-start.

Không xét torque, actuator, controller, collision, payload, compliance, backlash, sensor noise hoặc TCP đã hiệu chuẩn. Vì vậy success trong báo cáo là success của numerical IK trong MuJoCo model, không phải chứng nhận độ chính xác robot vật lý.

2.3. Giả thuyết nghiên cứu

Giả thuyết chính

Với cùng target, cùng q_{initial} , cùng DLS config và cùng budget, một policy PPO học bounded residual có thể tạo q_{seed} làm tăng xác suất hội tụ hoặc giảm số iteration của DLS, trong khi safety filter giữ degradation rate ở mức rất thấp.

Ba giả thuyết kiểm chứng:

$$H_1 : \text{Success}_{\text{SR-PPO-DLS}} > \text{Success}_{\text{DLS}}, \quad (4)$$

$$H_2 : \text{Stagnation}_{\text{SR-PPO-DLS}} < \text{Stagnation}_{\text{DLS}}, \quad (5)$$

$$H_3 : \text{DegradationRate} = P(S_B = 1, S_H = 0) \text{ gần } 0. \quad (6)$$

3. Nền tảng toán học của DLS

3.1. Pose error

Sai số vị trí được định nghĩa trong world frame:

$$e_p(q) = p_d - p(q) \in \mathbb{R}^3. \quad (7)$$

Sai số orientation dùng log map trên $\text{SO}(3)$:

$$e_o(q) = \text{Log}(R_d R(q)^T)^\vee \in \mathbb{R}^3. \quad (8)$$

Norm của e_o bằng geodesic angle giữa orientation hiện tại và orientation đích:

$$\|e_o(q)\|_2 = \theta \in [0, \pi]. \quad (9)$$

Ghép hai phần thành task error sáu chiều:

$$e(q) = \begin{bmatrix} e_p(q) \\ e_o(q) \end{bmatrix} \in \mathbb{R}^6. \quad (10)$$

3.2. Jacobian và xấp xỉ cục bộ

Geometric Jacobian của KR810 có kích thước:

$$J(q) = \begin{bmatrix} J_v(q) \\ J_\omega(q) \end{bmatrix} \in \mathbb{R}^{6 \times 7}. \quad (11)$$

Với thay đổi nhỏ Δq , chuyển động task-space được xấp xỉ bậc nhất:

$$\Delta x \approx J(q)\Delta q. \quad (12)$$

Đây là nguyên nhân DLS là phương pháp local: Jacobian chỉ mô tả tốt vùng lân cận của q hiện tại. Nếu seed ở basin không thuận lợi hoặc target xa, chuỗi cập nhật có thể giảm lỗi chậm, chậm limit hoặc stagnate.

3.3. Bài toán tối ưu DLS

DLS tìm bước Δq bằng cách cân bằng task error và joint-step norm:

$$\Delta q^* = \arg \min_{\Delta q} \left\| W^{1/2}(J\Delta q - e) \right\|_2^2 + \lambda^2 \|\Delta q\|_2^2, \quad (13)$$

trong đó $W \succeq 0$ là trọng số position/orientation và $\lambda > 0$ là damping. Điều kiện cực tiểu cho normal equation:

$$(J^T W J + \lambda^2 I) \Delta q = J^T W e \quad (14)$$

Solver dùng linear solve thay vì tính inverse trực tiếp. Damping làm giảm hệ số khuếch đại của các singular direction. Với SVD $J = U\Sigma V^T$, hệ số DLS theo singular value σ_i có dạng:

$$\frac{\sigma_i}{\sigma_i^2 + \lambda^2}, \quad (15)$$

thay cho $1/\sigma_i$ của pseudoinverse. Khi $\sigma_i \rightarrow 0$, hệ số DLS vẫn hữu hạn, giúp ổn định gần singularity [6].

3.4. Adaptive damping và giới hạn bước

Damping được tăng khi minimum singular value giảm:

$$\lambda(q) = f(\sigma_{\min}(J(q))), \quad \frac{\partial \lambda}{\partial \sigma_{\min}} < 0 \text{ trong vùng near-singular.} \quad (16)$$

Sau khi giải, bước được clamp:

$$\Delta q_j \leftarrow \text{clip}(\Delta q_j, -\Delta q_{\max,j}, \Delta q_{\max,j}), \quad (17)$$

$$q^{(i+1)} = \text{clip}(q^{(i)} + \Delta q, q_{\min}, q_{\max}). \quad (18)$$

Locked baseline hiện tại là `cand_D_pure_dls`: adaptive damping, tối đa 300 iterations, max joint step 0.1 rad, clip operational limits, joint-limit avoidance/null-space centering tắt. Solver convergence dùng strict 1 mm/1°; reporting còn có coarse 6 mm/5° và standard 3 mm/2° [7, 8].

3.5. Tại sao PPO cần cải thiện seed thay vì thay DLS

Nếu DLS đã ở trong basin tốt, nó có độ chính xác và numerical safety cao. Dataset v2 xác nhận 0 non-finite trên frozen workload. Vấn đề nổi bật là:

- cold-start thấp hơn warm-start rõ rệt;
- stagnation áp đảo max-iteration;
- failure tails rất lớn, cho thấy solver đôi khi dừng ở basin hoàn toàn sai;
- tăng iteration budget đơn thuần không giải quyết descent plateau.

Do đó, một policy học seed tốt có ý nghĩa hơn một policy học toàn bộ IK mapping.

4. Nền tảng toán học của PPO

4.1. MDP cho one-shot initializer

Một task được mô tả bởi context:

$$\mathcal{T} = (q_{\text{initial}}, p_d, R_d, \text{diagnostics}). \quad (19)$$

Episode one-shot có một state s , một action a , một DLS execution và một reward r . Vì chỉ có một bước, bài toán gần contextual bandit; tuy nhiên PPO vẫn hữu ích vì:

- action là continuous 7D;
- policy cần stochastic exploration;
- PPO-Clip cho phép nhiều minibatch epochs trên rollout;
- cùng policy architecture có thể được giữ ổn định khi mở rộng observation hoặc curriculum.

4.2. Actor-Critic

Actor tham số θ định nghĩa policy:

$$\pi_{\theta}(a \mid s). \quad (20)$$

Với continuous action, một lựa chọn tiêu chuẩn là diagonal Gaussian:

$$u \sim \mathcal{N}(\mu_{\theta}(s), \text{diag}(\sigma_{\theta}^2)), \quad (21)$$

$$a = \text{clip}(u, -1, 1) \in [-1, 1]^7. \quad (22)$$

Critic tham số ϕ ước lượng:

$$V_{\phi}(s) \approx \mathbb{E}[G \mid s]. \quad (23)$$

Trong deterministic evaluation, dùng mean action $a = \text{clip}(\mu_{\theta}(s), -1, 1)$.

Lưu ý triển khai Gaussian

Nếu dùng action clipping như Stable-Baselines3 PPO mặc định, log-probability phải được tính trên action trước clipping. Tỷ lệ action bị clip cần được log; clipping quá nhiều là dấu hiệu action scale hoặc policy variance chưa phù hợp. Nếu dùng tanh-squashed Gaussian tùy biến, phải có log-probability correction.

4.3. PPO clipped objective

Policy ratio giữa policy mới và policy dùng để thu rollout:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (24)$$

Clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]. \quad (25)$$

Clip range ϵ hạn chế policy update quá lớn. PPO alternates giữa data collection và nhiều epochs minibatch optimization, tạo cân bằng tốt giữa độ đơn giản và sample efficiency [2].

4.4. Advantage và value loss

TD residual:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \quad (26)$$

Generalized Advantage Estimation:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma \lambda_{\text{GAE}})^l \delta_{t+l}. \quad (27)$$

Với one-shot episode và terminal sau action:

$$\hat{A} = r - V_\phi(s), \quad G = r. \quad (28)$$

Loss tổng dùng khi tối thiểu hóa:

$$\mathcal{L}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_V \mathbb{E}(V_\phi(s) - G)^2 - c_H \mathbb{E}[\mathcal{H}(\pi_\theta(\cdot | s))]. \quad (29)$$

Entropy bonus duy trì exploration; value loss giúp giảm variance.

5. Nguyên lý Residual và Safe trong SR-PPO-DLS

5.1. Residual reinforcement learning

Residual RL phân rã điều khiển thành phần đã biết và phần học:

$$u = u_{\text{base}} + u_{\text{RL}}. \quad (30)$$

Johannink et al. dùng RL để học phần residual bổ sung cho conventional controller, thay vì buộc RL học lại toàn bộ cấu trúc đã được controller giải tốt [3]. Trong SR-PPO-DLS, sự phân rã được điều chỉnh ở **mức seed**:

$$q_{\text{seed}} = q_{\text{initial}} + \Delta q_{\text{PPO}}. \quad (31)$$

DLS không phải thành phần cộng trực tiếp vào action cùng thời điểm; DLS là downstream optimizer dùng seed đã chỉnh. Tinh thần residual vẫn giữ nguyên: học phần hiệu chỉnh nhỏ, bảo toàn solver cổ điển mạnh.

5.2. Safety layer và action projection

Dalal et al. đề xuất thêm safety layer vào policy để sửa action trước khi áp dụng, bảo đảm ràng buộc continuous-action không bị vi phạm [4]. SR-PPO-DLS áp dụng nguyên tắc này theo hai tầng:

1. **Hard projection:** đưa seed vào operational joint range có safety margin.
2. **Acceptance gate:** so sánh proposal với seed gốc bằng diagnostics chỉ sử dụng thông tin runtime; nếu proposal rủi ro, fallback về q_{initial} .

Đây là adaptation cho IK, không phải safety layer đóng-form của Dalal. Nó không bảo đảm safety của robot vật lý; nó chỉ bảo đảm các ràng buộc kinematic và do-no-harm tương đối trong simulation.

5.3. Tại sao cần safety filter khi pure DLS đã mạnh

Vì Point-IK standard success frozen đã khoảng 95%, một policy không có filter có thể tăng success trên một số case khó nhưng làm hỏng nhiều case DLS vốn giải được. Do đó metric quan trọng là degradation rate:

$$\text{DegradationRate} = \frac{\#\{S_B = 1, S_H = 0\}}{N}. \quad (32)$$

Safety filter hỗ trợ policy học “không làm gì” khi proposal không tạo lợi ích đáng tin cậy.

6. Kiến trúc SR-PPO-DLS được đề xuất

6.1. Sơ đồ tổng thể

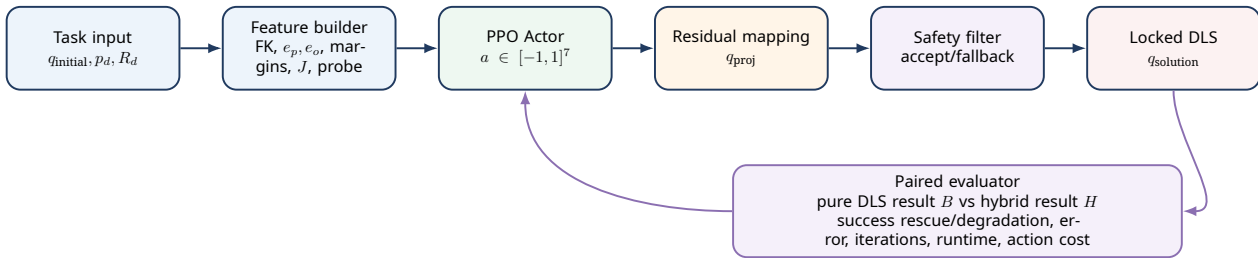


Figure 1: Kiến trúc SR-PPO-DLS. Chỉ seed thay đổi; locked DLS và target không đổi.

6.2. Một episode one-shot

1. Sample một task public: $(q_{\text{initial}}, p_d, R_d)$.
2. Tính observation s .
3. Actor sample action a .
4. Map action thành bounded residual Δq_{PPO} .
5. Project candidate vào safe operational range.
6. Safety filter accept proposal hoặc fallback.
7. Chạy locked DLS từ q_{seed} .
8. So sánh với pure DLS baseline của cùng task.
9. Tính reward và kết thúc episode.

6.3. Observation 36 chiều

Observation cuối cùng được đề xuất:

$$s = [\bar{q}, \bar{e}_p, \bar{e}_o, m^L, m^U, \bar{\sigma}_{\min}, \overline{\log(1 + \kappa)}, \Delta q_{\text{probe}}] \in \mathbb{R}^{36}. \quad (33)$$

Nhóm feature	Dim	Công thức/chuẩn hóa	Vai trò
Joint state	7	$\bar{q}_j = 2(q_j - q_{j,\min}) / (q_{j,\max} - q_{j,\min}) - 1$	Cho policy biết posture và vị trí trong joint range.
Position error	3	$\bar{e}_p = e_p / s_p$	Hướng và độ lớn sai số tịnh tiến.
Orientation error	3	$\bar{e}_o = e_o / \pi$	Hướng xoay ngắn nhất tới target.
Lower margins	7	$m_j^L = (q_j - q_{j,\min}) / (q_{j,\max} - q_{j,\min})$	Biết khớp nào gần lower limit.
Upper margins	7	$m_j^U = (q_{j,\max} - q_j) / (q_{j,\max} - q_{j,\min})$	Biết khớp nào gần upper limit.
Minimum singular value	1	chuẩn hóa bằng statistics training	Cho biết mức gần singularity.
Condition number	1	$\log(1 + \kappa)$ rồi chuẩn hóa	Đo anisotropy/ill-conditioning của Jacobian.
DLS probe direction	7	một DLS update chưa áp dụng, chia max joint step	Cho policy biết DLS hiện muốn đi hướng nào.

Các field bị cấm

Không đưa `q_target_reference`, `q_reference`, `waypoint_reachable`, `future joint solution` hoặc `protected conditioning fields` vào observation. Chúng chỉ dùng chứng minh target reachable. Dùng chúng sẽ tạo label leakage và làm phép so sánh với DLS không còn hợp lệ [7, 8].

6.4. DLS probe direction

Probe dùng đúng DLS single update tại seed gốc:

$$\Delta q_{\text{probe}} = (J^T W J + \lambda^2 I)^{-1} J^T W e. \quad (34)$$

Nhưng probe không được áp dụng trực tiếp. Nó là feature giúp Actor học residual so với hướng mà DLS đang đề xuất. Đây là cách giảm độ khó học: policy không cần tự học hoàn toàn inverse Jacobian từ đầu.

6.5. Action residual

Actor xuất normalized action:

$$a \in [-1, 1]^7. \quad (35)$$

Residual theo range của từng khớp:

$$\Delta q_j^{\text{PPO}} = \beta_j (q_{j,\max} - q_{j,\min}) a_j. \quad (36)$$

Ban đầu có thể dùng chung $\beta_j = \beta$. Candidate seed:

$$q_{\text{cand}} = q_{\text{initial}} + \Delta q^{\text{PPO}}. \quad (37)$$

Action bằng zero luôn hợp lệ:

$$a = 0 \Rightarrow q_{\text{cand}} = q_{\text{initial}}. \quad (38)$$

Đây là điều kiện quan trọng để policy học do-no-harm.

6.6. Projection vào safe joint range

Đặt safety margin $\delta_j \geq 0$:

$$q_{\text{proj},j} = \min(q_{j,\text{max}} - \delta_j, \max(q_{j,\text{min}} + \delta_j, q_{\text{cand},j})). \quad (39)$$

Projection bảo đảm proposal hữu hạn và nằm trong operational range. Không nên dùng một margin radian giống nhau cho mọi khớp nếu ranges khác đáng kể; dùng tỷ lệ range hoặc config đã hiệu chuẩn trên development.

6.7. Safety filter hai tầng

6.7.1. Tầng 1: hard feasibility

Reject proposal nếu:

- có NaN/Inf;
- FK hoặc Jacobian không hữu hạn;
- minimum joint-limit margin nhỏ hơn hard floor;
- projection distance quá lớn, cho thấy action thường xuyên đập vào boundary.

6.7.2. Tầng 2: one-step probe gate

Định nghĩa normalized weighted error:

$$E(q) = w_p \frac{\|e_p(q)\|}{s_p} + w_o \frac{\|e_o(q)\|}{s_o}, \quad w_p + w_o = 1. \quad (40)$$

Chạy cùng một micro-probe từ hai seed:

$$q_B^+ = \text{DLS}_1(q_{\text{initial}}, T_d), \quad (41)$$

$$q_P^+ = \text{DLS}_1(q_{\text{proj}}, T_d). \quad (42)$$

Accept proposal khi:

$$E(q_P^+) \leq E(q_B^+) + \tau \quad (43)$$

và margin không xấu quá ngưỡng. Seed cuối:

$$q_{\text{seed}} = \begin{cases} q_{\text{proj}}, & \text{nếu proposal được chấp nhận,} \\ q_{\text{initial}}, & \text{nếu fallback.} \end{cases} \quad (44)$$

$\tau \geq 0$ cho phép proposal tạm thời không tốt hơn local error một chút, vì một seed tốt hơn về basin không nhất thiết giảm error ngay lập tức. τ phải được tune trên development và khóa trước validation.

Chi phí của safety probe

Safety probe tạo thêm compute. Báo cáo runtime phải tách: PPO inference, two-seed probe cost và full DLS cost. Không được báo chỉ runtime DLS rồi bỏ qua chi phí filter.

7. Reward paired relative to pure DLS

7.1. Tại sao không dùng reward tuyệt đối

Nếu reward chỉ là $-E_H$, PPO có thể nhận reward tốt khi DLS tự giải được task, dù action không đóng góp. Reward phải so cùng task giữa:

$$B = \text{DLS}(q_{\text{initial}}, T_d), \quad (45)$$

$$H = \text{DLS}(q_{\text{seed}}, T_d). \quad (46)$$

Trong đó B là baseline cache và H là hybrid result.

7.2. Success indicators

Với reporting tier $k \in \{\text{coarse}, \text{standard}, \text{strict}\}$:

$$S_k = \mathbb{I}[e_p \leq \varepsilon_{p,k} \wedge e_o \leq \varepsilon_{o,k}]. \quad (47)$$

Standard 3 mm/2° là primary metric; strict 1 mm/1° là secondary precision metric.

7.3. Reward đề xuất

Một reward rõ attribution:

$$r = c_R(1 - S_B)S_H - c_D S_B(1 - S_H) \quad (48)$$

$$+ c_E \text{clip}(E_B - E_H, -1, 1) \quad (49)$$

$$+ c_I \mathbb{I}[S_B = S_H = 1] \text{clip}\left(\frac{I_B - I_H}{I_{\max}}, -1, 1\right) \quad (50)$$

$$- c_a \|a\|_2^2 - c_m \phi_m(q_{\text{seed}}) - c_\sigma \phi_\sigma(q_{\text{seed}}) - c_F \mathbb{I}[\text{fallback}]. \quad (51)$$

Giải thích:

- c_R : rescue bonus khi pure DLS fail nhưng hybrid success.
- c_D : degradation penalty khi pure DLS success nhưng hybrid fail; nên lớn hơn c_R .
- c_E : thưởng giảm normalized final error.
- c_I : thưởng giảm iteration chỉ khi cả hai cùng success, tránh thưởng dừng sớm do stagnation.
- c_a : regularize action nhỏ.
- ϕ_m : penalty gần joint limit.
- ϕ_σ : penalty nhẹ near-singular; không quá mạnh vì một số target hợp lệ có thể yêu cầu đi qua vùng xấu.
- fallback penalty nhỏ giúp policy giảm proposal vô ích nhưng không được lớn tới mức ép accept proposal nguy hiểm.

Một hàm limit penalty khả vi:

$$\phi_m(q) = \frac{1}{7} \sum_{j=1}^7 \left[\max\left(0, \frac{m_{\text{safe}} - m_j(q)}{m_{\text{safe}}}\right) \right]^2. \quad (52)$$

Singularity penalty:

$$\phi_\sigma(q) = \left[\max\left(0, \frac{\sigma_{\text{safe}} - \sigma_{\min}(q)}{\sigma_{\text{safe}}}\right) \right]^2. \quad (53)$$

7.4. Reward weights là hyperparameter, không phải kết luận lý thuyết

Không chốt số cuối trong tài liệu. Development search cần tune ít nhất:

$$\{c_R, c_D, c_E, c_I, c_a, c_m, c_\sigma, c_F, \beta, \tau\}. \quad (54)$$

TPE là lựa chọn phù hợp vì đã cho thấy cải thiện đáng kể với PPO trên robot 7-DOF [5]. Tuy nhiên search space và objective phải pre-register, không dùng frozen.

8. Quy trình huấn luyện PPO

8.1. Training stream và fixed benchmark

Dataset v2 public có 1,200 development Point-IK samples. Lập vô hạn trên 1,200 task có thể dẫn đến memorization. Phương án tốt nhất:

- giữ Dataset v2 development/validation/frozen làm fixed benchmark;
- tạo **SR-PPO training stream riêng** bằng cùng target-generation logic, seed namespace mới;
- kiểm tra content hash không trùng cả ba fixed splits;
- không lưu hoặc không expose $q_{\text{target_reference}}$ cho environment.

Training stream không làm thay đổi Dataset v2.0.0; nó là dữ liệu học riêng của method.

8.2. Sampling theo failure mode

Uniform sampling có thể bị easy cases chi phối. Curriculum đề xuất:

Stage	Task distribution	Mục tiêu
0	near-target, easy, pure DLS success nhanh	Học $a \approx 0$, giảm degradation.
1	pure DLS success nhưng iteration cao	Giữ success, giảm iterations/runtime.
2	stagnation, far-target, large-orientation, hard initial	Học rescue local basin.
3	cân bằng 6 difficulty groups	Generalization toàn Point-IK distribution.
4	waypoint 0 của trajectory development	Kiểm tra initial branch ảnh hưởng toàn đường.

Stage progression dùng stage-local budget và gate, không dùng global timestep đơn thuần. Có rollback nếu stage mới làm degradation tăng.

8.3. PPO training loop

```
for update in range(num_updates):
    rollout = []
    for env_step in range(rollout_size):
        task = sample_training_task()
        baseline = cached_pure_dls(task)
        s = build_observation(task)
```

```

u, log_prob, value = actor_critic.sample(s)
a = clip(u, -1, 1)
q_proj = residual_projection(task.q_initial, a)
q_seed, fallback = safety_filter(task, q_proj)
hybrid = locked_dls(task.target_pose, q_seed)
reward = paired_reward(baseline, hybrid, a, fallback)
rollout.append(s, u, log_prob, value, reward)

advantage, returns = one_step_advantage(rollout)
for epoch in range(ppo_epochs):
    for minibatch in shuffle(rollout):
        optimize_clipped_policy_value_entropy(minibatch)

```

8.4. Cấu hình khởi đầu

Tham số	Giá trị khởi đầu	Ghi chú
Policy	PPO-Clip	Theo Schulman et al.
Actor/Critic MLP	[256, 256]	Separate networks khuyến nghị.
Activation	Tanh	Phù hợp normalized state.
Learning rate	3×10^{-4}	Anneal/TPE sau smoke.
Clip range	0.2	Search 0.1–0.3.
Rollout size	4096	Tổng trên parallel envs.
Batch size	256	Phải chia hết rollout.
Epochs/rollout	10	Theo dõi approximate KL.
γ	1.0	One-shot terminal.
GAE λ	1.0	One-step advantage.
Entropy coefficient	10^{-3}	Anneal/TPE.
Value coefficient	0.5	Cấu hình khởi đầu.
Max grad norm	0.5	Ổn định update.
Parallel envs	8–16	DLS chủ yếu CPU.
Training steps	600k khởi đầu	So với paper MPDIK; không phải đảm bảo đủ.
Seeds	5	Báo mean, std và CI.

8.5. TPE hyperparameter optimization

Search trên training/development, không dùng validation/frozen:

- learning rate: 10^{-5} đến 10^{-3} log-uniform;
- action scale β : 0.01 đến 0.12;
- clip range: 0.1 đến 0.3;
- entropy: 10^{-5} đến 10^{-2} ;
- hidden width: 128, 256, 512;
- batch: 128, 256, 512;
- safety tolerance τ ;
- reward weights có constraint $c_D > c_R$.

Shianifar et al. báo TPE cải thiện success của PPO 34.28 percentage points và rút ngắn thời gian đạt 95% maximum reward 76% trong bài toán robot 7-DOF; kết quả này hỗ trợ quyết định dùng systematic search thay vì hand-tuning [5].

9. Áp dụng vào Dataset v2 hiện tại

9.1. Tài sản dữ liệu hiện có

Dataset v2.0.0 gồm:

- 2,600 Tier-0 states: 1,000 FK, 1,000 Jacobian, 600 singularity;
- 6,000 Point-IK: 1,200 development, 1,200 validation, 3,600 frozen-test;
- 12 anchors: 6 regular, 3 near-limit, 3 near-singular;
- 120 core trajectories và 90 random-challenge trajectories;
- 210 trajectories, 400 waypoints mỗi trajectory, tổng 84,000 canonical poses;
- 630 trials: easy, medium, hard; 210 trials mỗi split.

Các protected references bị loại khỏi public evaluation bundle, phù hợp với yêu cầu chống leakage [7, 8].

9.2. Mapping field cho Point-IK environment

Environment chỉ đọc public fields:

Field	Cách dùng	Ghi chú
q_initial	state khởi đầu	Không thay đổi giữa pure DLS và hybrid.
target_position	p_d	Target Cartesian reachable.
target_quaternion_wxyz	R_d	Chuyển quaternion sang rotation matrix.
difficulty_id/name	stratified sampling/report	Không đưa vào policy ở baseline đầu, trừ ablation.
sample_id	paired join	Bảo đảm so cùng task.
Public covariates	phân tích, chuẩn hóa	Không dùng field có nguồn từ protected solution nếu export policy cấm.

Tuyệt đối không đọc protected $q_{\text{target_reference}}$. Target đã được tạo bằng FK của một q hợp lệ, nhưng policy chỉ nhìn pose target giống numerical IK thực tế.

9.3. Vai trò của từng split

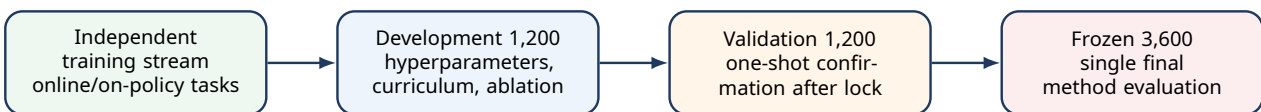


Figure 2: Protocol đề xuất cho SR-PPO-DLS.

1. **Training stream:** update PPO weights.

2. **Development:** TPE, reward tuning, action scale, safety threshold, checkpoint selection, ablation.
3. **Validation:** chạy xác nhận đúng một cấu hình đã khóa; không retune.
4. **Frozen:** sau code/config/model fingerprint lock, chạy final paired evaluation. Existing pure-DLS frozen artifact được dùng làm baseline cố định.

Vì frozen public release đã tồn tại, project phải tự enforce method-specific no-retune: ghi lock bundle trước lần chạy SR-PPO-DLS frozen, log access và cấm mọi thay đổi sau khi xem kết quả.

9.4. Primary application: Point-IK

Mỗi public Point-IK sample tạo một one-shot episode. Paired evaluation:

$$(q_{\text{initial}}, T_d) \xrightarrow{\text{pure DLS}} B, \quad (55)$$

$$(q_{\text{initial}}, T_d) \xrightarrow{\text{PPO+filter}} q_{\text{seed}} \xrightarrow{\text{same DLS}} H. \quad (56)$$

Chỉ khác seed. DLS config, target, tolerance và max iterations giống nhau.

9.5. Secondary application: trajectory waypoint 0

SR-PPO-DLS là initializer, nên trajectory application phù hợp nhất là:

$$q_{\text{initial}} \xrightarrow{\text{SR-PPO-DLS at waypoint 0}} q_0, \quad (57)$$

$$q_k = \text{DLS}_{\text{warm}}(q_{k-1}, T_{d,k}), \quad k = 1, \dots, 399. \quad (58)$$

Không gọi PPO lại ở waypoint 1–399. Thiết kế này trả lời câu hỏi sạch:

Một seed/branch tốt hơn tại waypoint đầu có cải thiện toàn bộ chuỗi warm-start hay không?

Mỗi split có 70 trajectories và 210 easy/medium/hard trials. So sánh:

- baseline warm-start DLS;
- SR-PPO-DLS tại waypoint 0 + warm-start DLS cho phần còn lại.

Không thay target trajectory, waypoint count, q_{initial} hoặc reporting thresholds.

9.6. Tại sao không PPO ở mọi waypoint

Gọi PPO mỗi waypoint sẽ thay đổi bài toán từ initializer thành repeated residual controller, tăng compute và làm attribution khó. User đã chốt không phát triển Multiple PPO, vì vậy method chính thức chỉ can thiệp một lần tại task start/trajectory start.

10. Thiết kế thực nghiệm và đánh giá công bằng

10.1. Baselines bắt buộc

Method	Mục đích
Pure DLS	Locked official baseline.
Random residual + DLS	Kiểm tra lợi ích có chỉ đến từ perturbation ngẫu nhiên.
DLS-probe heuristic seed + DLS	Baseline không học, dùng chính probe direction.
PPO residual không safety filter + DLS	Ablation của thành phần Safe.
PPO residual không DLS probe feature + DLS	Ablation observation.
SR-PPO-DLS full	Phương pháp đề xuất.

10.2. Point-IK metrics

Báo cáo theo split và 6 difficulty groups:

- coarse/standard/strict success rate;
- rescue count và degradation count;
- net success gain;
- position/orientation median, P95, max;
- DLS iterations median, P95;
- failure reason counts, đặc biệt stagnation;
- PPO action norm và clipping rate;
- safety fallback rate;
- minimum joint-limit margin và $\sigma_{\min}$;
- runtime PPO, filter, DLS, total P50/P95/P99;
- non-finite count.

10.3. Trajectory metrics

Với waypoint-0 initializer:

- waypoint standard success rate;
- position/orientation RMSE, P95, max;
- first failure location và maximum failure streak;
- recovery rate;
- cross-track RMSE/P95;
- final progress ratio và coverage;
- path-length ratio;
- maximum joint jump, total joint variation, velocity/jerk;
- runtime total cho cả trajectory.

10.4. Paired statistics

Vì hai method dùng cùng sample, dùng paired analysis. Với success difference $d_i = S_{H,i} - S_{B,i}$:

$$\Delta\hat{p} = \frac{1}{N} \sum_{i=1}^N d_i. \quad (59)$$

Dùng paired bootstrap theo sample/trial để tạo 95% confidence interval. Với binary discordant pairs, McNemar test có thể kiểm tra rescue vs degradation:

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}, \quad (60)$$

trong đó n_{01} là baseline fail/hybrid success và n_{10} là baseline success/hybrid fail.

10.5. Acceptance gates đề xuất

Các gate phải được pre-register trên development trước validation:

1. standard success improvement có paired bootstrap CI 95% không cắt 0;
2. degradation rate dưới ngưỡng khóa;
3. rescue count lớn hơn degradation count với margin rõ;
4. không tăng non-finite;
5. runtime P99 và fallback rate nằm trong budget khóa;
6. improvement không chỉ đến từ một difficulty group duy nhất;
7. trajectory waypoint-0 experiment không làm joint jump hoặc smoothness xấu đáng kể.

Không dùng reward làm kết quả

Training reward chỉ là tín hiệu tối ưu. Kết luận khoa học phải dựa trên paired success, error tails, iterations, stagnation, runtime và trajectory metrics giống baseline DLS.

11. Truy vết: thành phần nào áp dụng từ bài báo nào?

Thành phần SR-PPO-DLS	Nguồn chính	Cách project áp dụng/điều chỉnh
PPO clipped objective	Schulman et al. [2]	Dùng PPO-Clip cho continuous 7D residual action, minibatch epochs và actor-critic.
PPO giúp DLS bằng initial value	Yu và Tan [1]	Giữ ý tưởng PPO cải thiện seed, DLS tinh chỉnh pose. Khác paper: project dùng one-shot, không alternating multistep.
Residual action	Johannink et al. [3]	Giữ base solver và học bounded correction. Điều chỉnh residual từ controller output sang joint-space seed.

Thành phần SR-PPO-DLS	Nguồn chính	Cách project áp dụng/điều chỉnh
Safety filter/action correction	Dalal et al. [4]	Kế thừa nguyên tắc safety layer. Project dùng joint projection + DLS probe acceptance/fallback, không dùng learned linear constraint model nguyên bản.
TPE hyperparameter optimization	Shianifar et al. [5]	Tune action scale, learning rate, entropy, clip range, network và safety tolerance trên development.
DLS regularization near singularity	Buss và Kim [6]	Giải thích damping filter singular directions; locked solver vẫn là thành phần precision.
Dataset split, anti-leakage, frozen protocol	Project Dataset v2 [7, 8]	Train/tune không dùng protected reference; paired evaluation trên public dev/val/frozen.

Tuyên bố nguồn đúng

Không có bài báo nào mô tả chính xác toàn bộ SR-PPO-DLS như tài liệu này. Phương pháp là một synthesis có truy vết: PPO từ Schulman, PPO-DLS initialization từ Yu và Tan, residual decomposition từ Johannink, safety action correction từ Dalal, TPE từ Shianifar, và protocol/dataset từ project KR810.

12. Kiến trúc source code đề xuất

Không sửa các module DLS/kinematics đã khóa. Thêm package mới:

```
mpdik_kassow/|—
  sr_ppo_dls/|
    |— observation.py|
    |— residual_action.py|
    |— safety_filter.py|
    |— paired_reward.py|
    |— baseline_cache.py|
    |— fingerprints.py|—
  envs/|
    |— kr810_sr_ppo_dls_env.py|—
  configs/|
    |— sr_ppo_dls_config.json|—
  pipelines/|
    |— build_sr_ppo_training_stream.py|
    |— cache_pure_dls_pointik.py|
    |— train_sr_ppo_dls.py|
    |— validate_sr_ppo_dls.py|
    |— evaluate_sr_ppo_dls_frozen.py|—
  evaluation_v2/|
    |— sr_ppo_point_eval.py|
    |— sr_ppo_trajectory_init_eval.py|—
  tests/
    |— test_sr_ppo_observation.py
    |— test_sr_ppo_safety_filter.py
    |— test_sr_ppo_no_protected_fields.py
```

```
├─ test_sr_ppo_paired_reward.py
├─ test_sr_ppo_dls_lock_integrity.py
```

12.1. Test bắt buộc

- observation shape = 36, finite, deterministic;
- action zero trả đúng q_{initial} ;
- projection không vượt operational limits;
- safety fallback luôn trả seed hợp lệ;
- protected keys bị runtime guard từ chối;
- DLS config hash trước/sau SR-PPO giống nhau;
- paired reward: rescue positive, degradation negative;
- cache baseline khớp direct DLS rerun;
- deterministic evaluation dùng Actor mean;
- frozen pipeline từ chối chạy nếu model/config chưa lock.

13. Quy trình triển khai theo giai đoạn

Phase	Công việc	Gate đầu ra
A	Aggregate DLS failures theo difficulty/init	Bảng stagnation, iterations, success, margin, $\sigma_{\min}$.
B	Build independent PPO training stream	Anti-leakage pass; targets reachable; protected fields absent.
C	Baseline cache	Recompute audit pass; config hash locked.
D	Environment + observation/action/filter	Unit tests pass; random policy không tạo non-finite.
E	Stage 0 do-no-harm training	Degradation thấp; action gần zero trên easy.
F	High-iteration/stagnation curriculum	Rescue tăng trên development.
G	TPE + multi-seed selection	Cấu hình ổn định, không chọn theo một seed.
H	Validation confirmation	Một model/config lock; không retune sau validation.
I	Trajectory waypoint-0 test	Đo ảnh hưởng tới 400-waypoint warm-start.
J	Frozen final	Paired report, fingerprints, no-retune statement.

14. Rủi ro, giới hạn và cách kiểm soát

14.1. One-step PPO có thể không cần critic mạnh

Vì episode chỉ một bước, PPO gần contextual bandit. Critic vẫn giúp baseline variance nhưng có thể không tạo lợi ích lớn. Cần ablation với:

- PPO full actor-critic;

- REINFORCE/bandit baseline;
- supervised ranking seed proposer nếu có label từ candidate search chỉ trên training stream.

Phương pháp đã chốt vẫn là PPO; ablation giúp chứng minh lựa chọn thuật toán.

14.2. Reward hacking qua fallback

Policy có thể tạo proposal tùy ý rồi dựa vào fallback để không bị phạt. Giải pháp:

- log fallback rate;
- penalty fallback nhỏ;
- gate checkpoint nếu fallback quá cao;
- đánh giá proposal quality trước filter như diagnostic.

14.3. Memorization

Lập 1,200 development tasks có thể khiến policy nhớ task. Giải pháp là independent online training stream, content-hash anti-leakage và random target generation theo cùng distribution.

14.4. Action scale quá nhỏ hoặc quá lớn

Nếu β quá nhỏ, policy không đổi basin. Nếu quá lớn, policy gây branch switching và projection clipping. Vì vậy β là hyperparameter trọng yếu và phải được TPE/tune theo rescue-degradation objective.

14.5. Safety filter quá bảo thủ

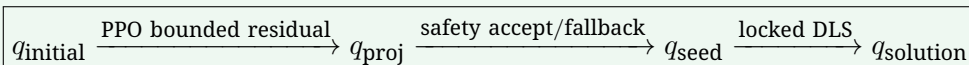
τ quá nhỏ khiến filter fallback gần như luôn, SR-PPO-DLS trở thành pure DLS. τ quá lớn giảm do-no-harm. Báo cáo phải công bố acceptance/fallback rate và improvement conditional on accepted proposals.

14.6. Không suy rộng sang robot thật

Ngay cả khi SR-PPO-DLS tăng numerical IK success, robot thật còn phụ thuộc calibration, actuator, controller, payload, speed và collision. Phương pháp hiện tại không giải các vấn đề đó.

15. Kết luận thiết kế

Kiến trúc cuối cùng



SR-PPO-DLS phù hợp với trạng thái project vì:

1. locked DLS đã được xác minh và rất mạnh ở Point-IK;
2. Dataset v2 cho thấy initialization sensitivity và stagnation là khoảng trống định lượng rõ;

3. residual action giảm độ khó so với PPO-only;
4. safety filter bảo vệ các case DLS vốn thành công;
5. paired reward cô lập đóng góp thật của PPO;
6. public/protected split và frozen protocol cho phép evaluation công bằng.

Phương pháp được áp dụng trước ở 6,000 Point-IK benchmark và sau đó ở waypoint đầu của mỗi trajectory. PPO không được gọi lặp lại dọc 400 waypoint; phần còn lại tiếp tục warm-start locked DLS. Kết quả chỉ được chấp nhận khi rescue tăng, degradation thấp, runtime được báo đầy đủ và improvement tồn tại trên validation/frozen, không chỉ trên training reward.

References

- [1] S. Yu and G. Tan, “Inverse Kinematics of a 7-Degree-of-Freedom Robotic Arm Based on Deep Reinforcement Learning and Damped Least Squares,” *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3521539.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017, doi: 10.48550/arXiv.1707.06347.
- [3] T. Johannink et al., “Residual Reinforcement Learning for Robot Control,” *2019 International Conference on Robotics and Automation (ICRA)*, 2019, doi: 10.1109/ICRA.2019.8794127.
- [4] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa, “Safe Exploration in Continuous Action Spaces,” *arXiv preprint arXiv:1801.08757*, 2018, doi: 10.48550/arXiv.1801.08757.
- [5] J. Shianifar, M. Schukat, and K. Mason, “Optimizing Deep Reinforcement Learning for Adaptive Robotic Arm Control,” in *Highlights in Practical Applications of Agents, Multi-Agent Systems, and Digital Twins: The PAAMS Collection*, Springer, 2025, pp. 293–304, doi: 10.1007/978-3-031-73058-0_24.
- [6] S. R. Buss and J.-S. Kim, “Selectively Damped Least Squares for Inverse Kinematics,” *Journal of Graphics Tools*, vol. 10, no. 3, pp. 37–49, 2005, doi: 10.1080/2151237X.2005.10129202.
- [7] MPDIK Kassow Project, “Current DLS Problem and Dataset Audit – Kassow KR810,” *CHATGPT_DLS_CURRENT_STATE_BRIEF.md*, branch `feature/dataset-v2`, commit `151ffd76`, 2026.
- [8] MPDIK Kassow Project, “Dataset v2 – Implementation Log,” *V2_IMPLEMENTATION_LOG.md*, 2026.